

2024 Sept OPPE 1 - SET 2

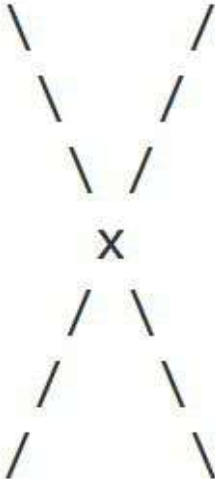
Section 3 - Problem 2

Problem Statement

Problem statement:

- You are given an integer n ($n \geq 0$). Your task is to print an “X” shaped pattern such that:
 - There are n rows above the center.
 - There are n rows below the center.
 - The center row contains only the character x .
 - The shape is formed using:
 - Backslash \
 - Forward slash /
- Spaces between the slashes change symmetrically as they move toward and away from the center.
- There must be no trailing spaces on any line.
- This is an I/O type question, so you must take input and print the pattern.

Example and Visualisation

n = 0	n = 1	n = 3
x		

Space Calculation Logic Concept

Understand the Pattern Structure

The pattern forms an "X" shape.

It consists of:

- n rows above the center,
- 1 middle row containing only x,
- n rows below the center
- The arms of the X are formed using \ and /
- Spaces between slashes increase as we move away from the center.

Algorithm for printing the Upper Half

- Loop from 0 to n-1.
- For each row:
 - Print leading spaces.
 - Print \
 - Print inner spaces (which decrease as we move downward).
 - Print /
- The distance between slashes gets smaller as we approach the center.

Printing the Center Row

- Print exactly:
 - n leading spaces
 - Followed by the character x
- This forms the exact center of the X.
- No trailing spaces should be printed.

Algorithm for printing the Lower Half

- Loop from n-1 down to 0.
- For each row:
 - Print leading spaces (decreasing as we go down).
 - Print /
 - Print inner spaces (increasing as we move away from center).
 - Print \
- This mirrors the upper half to complete the X shape.

Solution 1: Basic “for” Loop approach

```
n = int(input())

if n == 0:
    print("x")
else:
    # Upper half
    for i in range(n):
        print(" " * i + "\\ " + " " * (2*(n-i)-1) + "/" )    # spaces + "\ " + middle spaces + "/"
    # Center
    print(" " * n + "x")

    # Lower half
    for i in range(n-1, -1, -1): # reverses the upper half
        print(" " * i + "/" + " " * (2*(n-i)-1) + "\\ ")
```

(2 * (n-i)-1) formula for middle space:

decreases by 2 each row toward center AND
increases by 2 after center

The most beginner friendly, basic way to solve this problem

Solution 2: Compact Mathematical Approach

```
n = int(input())

if n == 0:
    print("x")
else:
    for i in range(-n, n+1):
        if i == 0:
            print(" " * n + "x")
        elif i < 0:
            k = -i
            print(" " * (n-k) + "\\\" + " " * (2*k-1) + "/"")
        else:
            print(" " * (n-i) + "/" + " " * (2*i-1) + "\\\"")
```

Single loop from **-n** to **n** controls the whole shape

for i in range(-n, n+1)

- Negative **i** → upper half
- **i == 0** → center
- Positive **i** → lower half
This avoids writing two separate loops.

A more mathematical approach to solve the problem

Solution comparison

Approach	Pros	Cons	Best For
Basic for Loop Approach	Highly readable, easy to debug, best for learning, clear variable names	More lines of code, repetition of logic	Educational purposes, beginners, complex debugging
Compact Mathematical Approach	Most concise, shorter and more efficient	Harder to debug, less readable for beginners, hard to understand	Experienced developers, simple contexts

Key Takeaways

- 🔑 Pattern problems require careful observation of symmetry. Practice is the most important thing to ace these type of problems.
- 🔑 The number of leading spaces increases toward the center and decreases afterward.
- 🔑 The inner spaces decrease toward the center and increase afterward.
- 🔑 Escape character `\\` must be used to print a backslash.
- 🔑 Handling the edge case ($n = 0$) is important.
- 🔑 Breaking the pattern into upper half, center, and lower half makes logic clear and manageable.